IPC SIMULATOR

PipeSimulation.java

```
package ipc;
import utils.Logger;
import java.io.*;
public class PipeSimulation {
    private static final String MODULE = "Pipes";
    public static void runSimulation() {
        Logger.logDivider("PIPE SIMULATION — REAL OS PROCESSES");
        Logger.logInfo(MODULE, "Spawning two real OS processes using ProcessBuilder...");
        try {
            String classpath = System.getProperty("java.class.path");
            // Spawn Producer process
            ProcessBuilder producerPB = new ProcessBuilder(
                "java", "-cp", classpath, "ipc.PipeProducer"
            );
            producerPB.redirectErrorStream(true);
            Process producer = producerPB.start();
            // Spawn Consumer process
            ProcessBuilder consumerPB = new ProcessBuilder(
                "java", "-cp", classpath, "ipc.PipeConsumer"
            );
            consumerPB.redirectErrorStream(true);
            Process consumer = consumerPB.start();
            long producerPID = producer.pid();
            long consumerPID = consumer.pid();
            Logger.logInfo(MODULE, "Producer process started — PID: " + producerPID);
            Logger.logInfo(MODULE, "Consumer process started — PID: " + consumerPID);
            Logger.logWarning(MODULE, "Tip: Open Task Manager and search these PIDs to
verify real OS processes!");
            // Thread: read producer stdout, log it, forward to consumer stdin
            new Thread(() -> {
```

```
            try (
                BufferedReader prodReader = new BufferedReader(
                    new InputStreamReader(producer.getInputStream()));
                OutputStream consumerIn = consumer.getOutputStream()
            ) {
                String line;
                while ((line = prodReader.readLine()) != null) {
                    if (!line.isEmpty()) {
                        Logger.logSent(MODULE, "[PID " + producerPID + "] → \"" + line +
"\"");
                        consumerIn.write((line + "\n").getBytes());
                        consumerIn.flush();
                        if (line.equals("END")) break;
                    }
                }
            } catch (IOException e) {
                Logger.logError(MODULE, "Pipe error: " + e.getMessage());
            }
        }).start();
        new Thread(() -> {
            try (BufferedReader consumerReader = new BufferedReader(
                    new InputStreamReader(consumer.getInputStream()))) {
                String line;
                while ((line = consumerReader.readLine()) != null) {
                    if (!line.isEmpty())
                        Logger.logReceived(MODULE, "[PID " + consumerPID + "] ← \"" + line +
"\"");
                }
            } catch (IOException e) {
                Logger.logError(MODULE, "Consumer read error: " + e.getMessage());
            }
        }).start();
        new Thread(() -> {
```

```
            try {
                producer.waitFor();
                consumer.waitFor();
                Logger.logInfo(MODULE, "Both OS processes exited. Real pipe closed.");
                Logger.logInfo(MODULE, "PIDs " + producerPID + " & " + consumerPID + "
are no longer active.");
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }).start();


    } catch (IOException e) {
        Logger.logError(MODULE, "ProcessBuilder failed: " + e.getMessage());
        Logger.logWarning(MODULE, "Falling back to thread simulation...");
        runFallback();
    }
  }
  private static void runFallback() {
    java.util.concurrent.LinkedBlockingQueue<String> pipe =
        new java.util.concurrent.LinkedBlockingQueue<>();
    Thread producer = new Thread(() -> {
        try {
            String[] messages = {"Hello from Producer!", "Data packet #1", "Data packet #2",
"END"};
            for (String msg : messages) {
                Thread.sleep(500);
                pipe.put(msg);
                Logger.logSent(MODULE, "Producer → \"" + msg + "\"");
            }
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    });
    Thread consumer = new Thread(() -> {
        try {
```

```
            while (true) {
                String msg = pipe.take();
                Logger.logReceived(MODULE, "Consumer ← \"" + msg + "\"");
                if (msg.equals("END")) break;
            }
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    });
    producer.start(); consumer.start();
  }
}
```

PipeProducer.java

```
package ipc;

public class PipeProducer {
  public static void main(String[] args) throws Exception {
    String[] messages = {
      "Hello from Producer!",
      "Data packet #1",
      "Data packet #2",
      "Data packet #3",
      "END"
    };
    for (String msg : messages) {
      Thread.sleep(600);
      System.out.println(msg);
      System.out.flush();
    }
  }
}
```

Pipeconsumer.java

```
package ipc;
```

```
import java.util.Scanner;

public class PipeConsumer {
    public static void main(String[] args) throws Exception {
        Scanner scanner = new Scanner(System.in);
        while (scanner.hasNextLine()) {
            String msg = scanner.nextLine().trim();
            if (!msg.isEmpty()) {
                System.out.println("Received: " + msg);
                System.out.flush();
                if (msg.equals("END")) break;
            }
        }
    }
}
```

MessageQueue.java

```
package ipc;

import utils.Logger;
import java.util.concurrent.LinkedBlockingQueue;

public class MessageQueue {
    private static final String MODULE = "MsgQueue";
    private static final LinkedBlockingQueue<String> queue = new
LinkedBlockingQueue<>(10);

    public static void runSimulation() {
        Logger.logDivider("MESSAGE QUEUE SIMULATION");
        Logger.logInfo(MODULE, "3 Producers sending to shared queue — 2 Consumers
reading (FIFO order)");

        queue.clear();
```

```
        // 3 producers
        for (int i = 1; i <= 3; i++) {
            final int id = i;
            new Thread(() -> {
                try {
                    for (int j = 1; j <= 2; j++) {
                        Thread.sleep((long)(Math.random() * 600 + 300));
                        String msg = "MSG-P" + id + "-#" + j;
                        queue.put(msg);
                        Logger.logSent(MODULE, "Producer-" + id + " enqueued → \"" + msg + "\" |
Queue size: " + queue.size());
                    }
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }).start();
        }

        // 2 consumers
        for (int i = 1; i <= 2; i++) {
            final int id = i;
            new Thread(() -> {
                try {
                    int count = 0;
                    while (count < 3) {
                        Thread.sleep(800);
                        String msg = queue.poll();
                        if (msg != null) {
                            Logger.logReceived(MODULE, "Consumer-" + id + " dequeued ← \"" +
msg + "\" | Queue size: " + queue.size());
                            count++;
                        }
                    }
```

```
                Logger.logInfo(MODULE, "Consumer-" + id + " finished processing.");
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }).start();
    }
  }
}
```

Deadlock.java

```
package ipc;

import utils.Logger;
import java.util.concurrent.locks.ReentrantLock;

public class Deadlock {
  private static final String MODULE = "Deadlock";
  private static final ReentrantLock lockA = new ReentrantLock();
  private static final ReentrantLock lockB = new ReentrantLock();

  public static void runSimulation() {
    Logger.logDivider("DEADLOCK DEMONSTRATION");
    Logger.logWarning(MODULE, "Thread-1 locks A then wants B. Thread-2 locks B then wants A → DEADLOCK!");
    runDeadlock();

    new Thread(() -> {
      try {
        Thread.sleep(4000);
        Logger.logDivider("DEADLOCK FIXED");
        Logger.logInfo(MODULE, "Both threads now acquire locks in same order (A then B) → No deadlock!");
        runFixed();
      } catch (InterruptedException e) {
```

```
            Thread.currentThread().interrupt();
        }
    }).start();
}

private static void runDeadlock() {
    Thread t1 = new Thread(() -> {
        try {
            lockA.lock();
            Logger.logError(MODULE, "Thread-1: acquired Lock-A, waiting for Lock-B...");
            Thread.sleep(500);
            // Try to get B (but T2 holds it)
            if (lockB.tryLock()) {
                Logger.logSent(MODULE, "Thread-1: acquired Lock-B (lucky, no deadlock this
run)");
                lockB.unlock();
            } else {
                Logger.logError(MODULE, "Thread-1: CANNOT acquire Lock-B →
DEADLOCK! Stuck forever.");
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } finally {
            if (lockA.isHeldByCurrentThread()) lockA.unlock();
        }
    });

    Thread t2 = new Thread(() -> {
        try {
            lockB.lock();
            Logger.logError(MODULE, "Thread-2: acquired Lock-B, waiting for Lock-A...");
            Thread.sleep(500);
            if (lockA.tryLock()) {
```

```
                Logger.logSent(MODULE, "Thread-2: acquired Lock-A (lucky, no deadlock this
run)");
                lockA.unlock();
            } else {
                Logger.logError(MODULE, "Thread-2: CANNOT acquire Lock-A → 
DEADLOCK! Stuck forever.");
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } finally {
            if (lockB.isHeldByCurrentThread()) lockB.unlock();
        }
    });

    t1.start(); t2.start();
  }

  private static void runFixed() {
    Thread t1 = new Thread(() -> {
        try {
            lockA.lock();
            Logger.logSent(MODULE, "Thread-1: acquired Lock-A");
            Thread.sleep(300);
            lockB.lock();
            Logger.logSent(MODULE, "Thread-1: acquired Lock-B → doing work safely ✓");
            Thread.sleep(300);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } finally {
            if (lockB.isHeldByCurrentThread()) lockB.unlock();
            if (lockA.isHeldByCurrentThread()) lockA.unlock();
            Logger.logInfo(MODULE, "Thread-1: released both locks.");
        }
```

```
        });

        Thread t2 = new Thread(() -> {
            try {
                Thread.sleep(100);
                lockA.lock(); // Same order as T1
                Logger.logReceived(MODULE, "Thread-2: acquired Lock-A");
                Thread.sleep(300);
                lockB.lock();
                Logger.logReceived(MODULE, "Thread-2: acquired Lock-B → doing work safely
✓");
                Thread.sleep(300);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            } finally {
                if (lockB.isHeldByCurrentThread()) lockB.unlock();
                if (lockA.isHeldByCurrentThread()) lockA.unlock();
                Logger.logInfo(MODULE, "Thread-2: released both locks. No deadlock!");
            }
        });

        t1.start(); t2.start();
    }
}
```

SemaphoreSimulation.java

```
package ipc;
import utils.Logger;
import java.util.concurrent.Semaphore;
public class SemaphoreSimulation {
    private static final String MODULE = "Semaphore";
    private static int resource = 0;
    public static void runSimulation() {
```

```
        Logger.logDivider("SEMAPHORE — WITHOUT (Race Condition)");
        Logger.logWarning(MODULE, "4 threads accessing shared resource WITHOUT
semaphore...");
        resource = 0;
        runWithoutSemaphore();
        new Thread(() -> {
            try {
                Thread.sleep(4000);
                Logger.logDivider("SEMAPHORE — WITH (Fixed)");
                Logger.logInfo(MODULE, "Same 4 threads WITH semaphore (only 1 at a
time)...");
                resource = 0;
                runWithSemaphore();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }).start();
    }
    private static void runWithoutSemaphore() {
        for (int i = 1; i <= 4; i++) {
            final int id = i;
            new Thread(() -> {
                try {
                    Logger.logError(MODULE, "Thread-" + id + " entering critical section!");
                    Thread.sleep(300);
                    resource++;
                    Logger.logError(MODULE, "Thread-" + id + " modified resource = " + resource
+ " (UNSAFE!)");
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }).start();
        }
```

```
    }
    private static void runWithSemaphore() {
        Semaphore semaphore = new Semaphore(1); // Only 1 thread at a time
        for (int i = 1; i <= 4; i++) {
            final int id = i;
            new Thread(() -> {
                try {
                    semaphore.acquire();
                    Logger.logSent(MODULE, "Thread-" + id + " acquired semaphore → entering
critical section");
                    Thread.sleep(400);
                    resource++;
                    Logger.logReceived(MODULE, "Thread-" + id + " modified resource = " +
resource + " (SAFE ✓)");
                    semaphore.release();
                    Logger.logInfo(MODULE, "Thread-" + id + " released semaphore");
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }).start();
        }
    }
}
```

SharedMemory.java

```
package ipc;
import utils.Logger;
public class SharedMemory {
    private static final String MODULE = "SharedMem";
    private static int sharedValue = 0;
    public static void runSimulation() {
        Logger.logDivider("SHARED MEMORY — WITHOUT SYNC (Race Condition)");
        Logger.logWarning(MODULE, "Two threads writing to shared memory WITHOUT
synchronization...");
```

```
    sharedValue = 0;
    runWithoutSync();
    // After delay, show fixed version
    new Thread(() -> {
        try {
            Thread.sleep(3500);
            Logger.logDivider("SHARED MEMORY — WITH SYNC (Fixed)");
            Logger.logInfo(MODULE, "Same scenario WITH synchronized blocks...");
            sharedValue = 0;
            runWithSync();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }).start();
}
private static void runWithoutSync() {
    Thread t1 = new Thread(() -> {
        for (int i = 0; i < 5; i++) {
            int temp = sharedValue;
            try { Thread.sleep(50); } catch (InterruptedException e) {}
            sharedValue = temp + 1;
            Logger.logError(MODULE, "Thread-1 wrote sharedValue = " + sharedValue);
        }
    });
    Thread t2 = new Thread(() -> {
        for (int i = 0; i < 5; i++) {
            int temp = sharedValue;
            try { Thread.sleep(50); } catch (InterruptedException e) {}
            sharedValue = temp + 1;
            Logger.logError(MODULE, "Thread-2 wrote sharedValue = " + sharedValue);
        }
    });
```

```
        t1.start(); t2.start();
        new Thread(() -> {
            try {
                t1.join(); t2.join();
                Logger.logWarning(MODULE, "Final sharedValue (expected 10, got) = " +
sharedValue + " ← RACE CONDITION!");
            } catch (InterruptedException e) {}
        }).start();
    }
    private static void runWithSync() {
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                synchronized (SharedMemory.class) {
                    int temp = sharedValue;
                    try { Thread.sleep(50); } catch (InterruptedException e) {}
                    sharedValue = temp + 1;
                    Logger.logSent(MODULE, "Thread-1 safely wrote sharedValue = " +
sharedValue);
                }
            }
        });
        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                synchronized (SharedMemory.class) {
                    int temp = sharedValue;
                    try { Thread.sleep(50); } catch (InterruptedException e) {}
                    sharedValue = temp + 1;
                    Logger.logReceived(MODULE, "Thread-2 safely wrote sharedValue = " +
sharedValue);
                }
            }
        });
        t1.start(); t2.start();
```

```
        new Thread(() -> {
            try {
                t1.join(); t2.join();
                Logger.logInfo(MODULE, "Final sharedValue (expected 10, got) = " +
sharedValue + " ✓ CORRECT!");
            } catch (InterruptedException e) {}
        }).start();
    }
}
```

Signals.java

```
package ipc;
import utils.Logger;
public class Signals {
    private static final String MODULE = "Signals";
    private static final Object lock = new Object();
    private static boolean signalSent = false;
    public static void runSimulation() {
        Logger.logDivider("SIGNALS SIMULATION");
        Logger.logInfo(MODULE, "Receiver waits for signal. Sender does work then signals
receiver.");
        signalSent = false;
        Thread receiver = new Thread(() -> {
            synchronized (lock) {
                try {
                    Logger.logWarning(MODULE, "Receiver: waiting for signal from Sender...");
                    while (!signalSent) lock.wait();
                    Logger.logReceived(MODULE, "Receiver: signal received! Resuming
execution.");
                    Logger.logInfo(MODULE, "Receiver: processing data after signal...");
                    Thread.sleep(500);
                    Logger.logInfo(MODULE, "Receiver: done.");
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
```

```
            }
        }
    });
    Thread sender = new Thread(() -> {
        try {
            Logger.logInfo(MODULE, "Sender: doing some work before signaling...");
            Thread.sleep(1000);
            Logger.logInfo(MODULE, "Sender: work done. Sending signal now...");
            Thread.sleep(500);
            synchronized (lock) {
                signalSent = true;
                lock.notifyAll();
                Logger.logSent(MODULE, "Sender: signal sent via notify()!");
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    });
    receiver.start();
    try { Thread.sleep(200); } catch (InterruptedException e) {}
    sender.start();
  }
}
```

SimulatorUI.java

```
package ui;
import ipc.*;
import utils.Logger;
import javax.swing.*;
import javax.swing.text.*;
import java.awt.*;
import java.awt.event.*;
public class SimulatorUI extends JFrame {
```

```
private JTextPane logPane;
private StyledDocument doc;
public SimulatorUI() {
    setTitle("IPC Simulator — Inter Process Communication");
    setSize(900, 620);
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout(0, 0));
    // Dark background
    Color bg        = new Color(18, 18, 28);
    Color panelBg   = new Color(26, 26, 40);
    Color accent    = new Color(80, 120, 255);
    Color btnBg     = new Color(36, 36, 54);
    Color btnHover  = new Color(55, 55, 80);
    Color textColor = new Color(200, 200, 220);
    getContentPane().setBackground(bg);
    // ── Title bar ──
    JPanel titleBar = new JPanel(new FlowLayout(FlowLayout.LEFT, 20, 12));
    titleBar.setBackground(new Color(22, 22, 35));
    titleBar.setBorder(BorderFactory.createMatteBorder(0, 0, 2, 0, accent));
    JLabel title = new JLabel("⚙  IPC Simulator");
    title.setFont(new Font("Monospaced", Font.BOLD, 18));
    title.setForeground(new Color(140, 170, 255));
    JLabel sub = new JLabel("Operating Systems — Inter Process Communication");
    sub.setFont(new Font("SansSerif", Font.PLAIN, 12));
    sub.setForeground(new Color(120, 120, 150));
    titleBar.add(title);
    titleBar.add(sub);
    add(titleBar, BorderLayout.NORTH);
    // ── Left button panel ──
    JPanel leftPanel = new JPanel();
    leftPanel.setLayout(new BoxLayout(leftPanel, BoxLayout.Y_AXIS));
```

```
leftPanel.setBackground(panelBg);
leftPanel.setBorder(BorderFactory.createMatteBorder(0, 0, 0, 2, new Color(40, 40,
60)));
leftPanel.setPreferredSize(new Dimension(190, 0));
JLabel menuLabel = new JLabel("  IPC MECHANISMS");
menuLabel.setFont(new Font("Monospaced", Font.BOLD, 11));
menuLabel.setForeground(new Color(90, 100, 140));
menuLabel.setBorder(BorderFactory.createEmptyBorder(16, 12, 10, 0));
leftPanel.add(menuLabel);
String[][] buttons = {
    {"🔵 Pipes",          "PIPE"},
    {"🟢 Message Queue",  "MQ"},
    {"🟡 Shared Memory",  "SHM"},
    {"🔴 Semaphores",     "SEM"},
    {"🟠 Signals",        "SIG"},
    {"🟣 Deadlock",       "DL"},
};
for (String[] btn : buttons) {
    JButton b = makeButton(btn[0], btn[1], btnBg, btnHover, textColor);
    leftPanel.add(b);
    leftPanel.add(Box.createVerticalStrut(4));
}

leftPanel.add(Box.createVerticalGlue());
// Clear button
JButton clearBtn = new JButton("🗑  Clear Log");
clearBtn.setFont(new Font("SansSerif", Font.PLAIN, 12));
clearBtn.setBackground(new Color(60, 30, 30));
clearBtn.setForeground(new Color(220, 120, 120));
clearBtn.setBorder(BorderFactory.createEmptyBorder(10, 16, 10, 16));
clearBtn.setFocusPainted(false);
clearBtn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
```

```
        clearBtn.setMaximumSize(new Dimension(190, 42));
        clearBtn.addActionListener(e -> {
            try { doc.remove(0, doc.getLength()); } catch (BadLocationException ex) {}
        });
        leftPanel.add(clearBtn);
        leftPanel.add(Box.createVerticalStrut(12));
        add(leftPanel, BorderLayout.WEST);
        // ── Log panel ──
        logPane = new JTextPane();
        logPane.setEditable(false);
        logPane.setBackground(new Color(14, 14, 22));
        logPane.setFont(new Font("Monospaced", Font.PLAIN, 13));
        logPane.setBorder(BorderFactory.createEmptyBorder(10, 14, 10, 14));
        doc = logPane.getStyledDocument();
        JScrollPane scroll = new JScrollPane(logPane);
        scroll.setBackground(bg);
        scroll.setBorder(BorderFactory.createEmptyBorder());
        scroll.getVerticalScrollBar().setBackground(new Color(30, 30, 45));
        add(scroll, BorderLayout.CENTER);
        // ── Status bar ──
        JPanel statusBar = new JPanel(new FlowLayout(FlowLayout.LEFT, 16, 6));
        statusBar.setBackground(new Color(20, 20, 32));
        statusBar.setBorder(BorderFactory.createMatteBorder(1, 0, 0, 0, new Color(40, 40,
60)));
        JLabel status = new JLabel("● Ready — Select an IPC mechanism to simulate");
        status.setFont(new Font("Monospaced", Font.PLAIN, 11));
        status.setForeground(new Color(80, 200, 120));
        statusBar.add(status);
        add(statusBar, BorderLayout.SOUTH);
        // Logger callback
        Logger.setCallback((msg, color) -> SwingUtilities.invokeLater(() -> {
            try {
```

```
            SimpleAttributeSet attr = new SimpleAttributeSet();
            StyleConstants.setForeground(attr, color);
            StyleConstants.setFontFamily(attr, "Monospaced");
            StyleConstants.setFontSize(attr, 13);
            doc.insertString(doc.getLength(), msg + "\n", attr);
            logPane.setCaretPosition(doc.getLength());
            status.setText("● Running simulation...");
            status.setForeground(new Color(255, 200, 60));
        } catch (BadLocationException e) {}
    }));
    setVisible(true);
    // Welcome message
    Logger.logDivider("IPC SIMULATOR STARTED");
    Logger.log("System", "Welcome! Select any IPC mechanism from the left panel.");
    Logger.log("System", "Each simulation runs in real-time with color-coded output.");
    Logger.logDivider("");
}
private JButton makeButton(String label, String type, Color bg, Color hover, Color fg) {
    JButton b = new JButton(label);
    b.setFont(new Font("SansSerif", Font.PLAIN, 13));
    b.setBackground(bg);
    b.setForeground(fg);
    b.setBorder(BorderFactory.createEmptyBorder(11, 16, 11, 16));
    b.setFocusPainted(false);
    b.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
    b.setMaximumSize(new Dimension(190, 46));
    b.setHorizontalAlignment(SwingConstants.LEFT);
    b.addMouseListener(new MouseAdapter() {
        public void mouseEntered(MouseEvent e) { b.setBackground(hover); }
        public void mouseExited(MouseEvent e)  { b.setBackground(bg); }
    });
    b.addActionListener(e -> {
```

```
            new Thread(() -> {
                switch (type) {
                    case "PIPE" -> PipeSimulation.runSimulation();
                    case "MQ"   -> MessageQueue.runSimulation();
                    case "SHM"  -> SharedMemory.runSimulation();
                    case "SEM"  -> SemaphoreSimulation.runSimulation();
                    case "SIG"  -> Signals.runSimulation();
                    case "DL"   -> Deadlock.runSimulation();
                }
            }).start();
        });
        return b;
    }
}
```

Logger.java

```
package utils;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.util.function.BiConsumer;
import java.awt.Color;
public class Logger {
    private static BiConsumer<String, Color> logCallback;
    public static final Color GREEN  = new Color(0, 180, 80);
    public static final Color BLUE   = new Color(30, 120, 255);
    public static final Color RED    = new Color(220, 50, 50);
    public static final Color ORANGE = new Color(220, 140, 0);
    public static final Color GRAY   = new Color(130, 130, 150);
    public static final Color WHITE  = new Color(220, 220, 230);
    public static void setCallback(BiConsumer<String, Color> callback) {
        logCallback = callback;
    }
    private static String getTimestamp() {
```

```
        return "[" + LocalTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss")) + "]";
    }
    public static void log(String module, String msg) {
        send(getTimestamp() + " " + module + " | " + msg, WHITE);
    }
    public static void logSent(String module, String msg) {
        send(getTimestamp() + " " + module + " | ➤ SENT: " + msg, GREEN);
    }
    public static void logReceived(String module, String msg) {
        send(getTimestamp() + " " + module + " | ✓ RECEIVED: " + msg, BLUE);
    }
    public static void logError(String module, String msg) {
        send(getTimestamp() + " " + module + " | ✕ " + msg, RED);
    }
    public static void logWarning(String module, String msg) {
        send(getTimestamp() + " " + module + " | ⚠ " + msg, ORANGE);
    }
    public static void logInfo(String module, String msg) {
        send(getTimestamp() + " " + module + " | ℹ " + msg, GRAY);
    }
    public static void logDivider(String label) {
        send("━━━━━━━━━━━━━━━━━━━━ " + label + " ━━━━━━━━━━━━━━━━━━━━", new Color(80,
80, 100));
    }
    private static void send(String msg, Color color) {
        if (logCallback != null) logCallback.accept(msg, color);
        else System.out.println(msg);
    }
}
```

Main.java

```
import ui.SimulatorUI;
import javax.swing.SwingUtilities;
```

```
public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(SimulatorUI::new);
    }
}
```

run.sh

```
#!/bin/bash
echo "Compiling IPC Simulator..."
mkdir -p out
javac -d out utils/Logger.java ipc/PipeProducer.java ipc/PipeConsumer.java
ipc/PipeSimulation.java ipc/MessageQueue.java ipc/SharedMemory.java
ipc/SemaphoreSimulation.java ipc/Signals.java ipc/Deadlock.java ipc/SocketIPC.java
ui/SimulatorUI.java Main.java
java -cp out Main
```